IT Business School (ITBS) – Nabeul

Module : Big Data – Mini-projet ELK & NoSQL

DOSSIER D'ANALYSE ET DE CONCEPTION

SaaSWatch

Besoins, architecture, modèle de données et maquettes

Scénario : D — Application Web SaaS

Réalisé par : Malek Mallat

Encadré par : M. Mohamed Najeh ISSAOUI

Sprint : 1 — Analyse et conception

Sommaire

1. Présentation et besoins 3
1.1 Contexte et problématique 3
1.2 Objectifs 3
1.3 Acteurs et indicateurs clés 3
1.4 Besoins fonctionnels 3
1.5 Besoins non fonctionnels 3
1.6 Contraintes techniques 4
2. Diagramme de cas d'utilisation 5
3. Architecture technique 6
3.1 Architecture globale (C4 – niveau conteneurs) 6
3.2 Diagramme de déploiement Docker 6
4. Diagrammes de séquence 8
4.1 Import d'un fichier de logs 8
4.2 Recherche de logs 9
4.3 Déclenchement d'une alerte 10
5. Modèle de données 11
5.1 Elasticsearch : index template logs-saas-* 11
5.2 MongoDB : collections 11
5.3 Redis : clés et usages 12
6. Maquettes de l'interface 13
6.1 Wireframes 13
6.2 Design system 15
6.3 Maquette finale : tableau de bord 16
6.4 Parcours utilisateur 16

1. Présentation et besoins

1.1 Contexte et problématique

Une application SaaS utilisée par des milliers d'entreprises produit en permanence des logs : serveur web, application, base de données et performances. Ces logs sont dispersés, leur analyse est manuelle, aucune alerte ne prévient l'équipe en cas de lenteur ou d'erreur, et il n'existe aucune vue d'ensemble.

1.2 Objectifs

SaaSWatch doit **centraliser** les logs dans Elasticsearch, permettre de les **rechercher** rapidement, **visualiser** les indicateurs clés, **alerter** l'équipe DevOps et afficher les événements **en temps réel**, le tout déployé avec Docker Compose. Les logs de test sont générés par un script Python (Faker) au format CSV et JSON.

1.3 Acteurs et indicateurs clés

Acteur	Besoin principal
Ingénieur DevOps	Surveiller l'application, recevoir et traiter les alertes
Développeur	Rechercher les erreurs d'un service pour corriger les bugs
Product manager	Suivre l'usage des fonctionnalités
Administrateur	Gérer les utilisateurs, les rôles et les règles d'alerte

KPI suivis : temps de réponse moyen par endpoint · taux d'erreurs 5xx par service · requêtes SQL les plus lentes · utilisateurs actifs · fonctionnalités les plus et les moins utilisées.

1.4 Besoins fonctionnels

Module	Besoins	Niveau
Gestion des fichiers	Import CSV/JSON/TXT par glisser-déposer, validation, prévisualisation, ingestion par Logstash, liste des fichiers, métadonnées dans MongoDB	Obligatoire
Recherche	Texte libre, filtres (niveau, service, endpoint, dates), tableau paginé et trié, détail d'un log, export CSV, historique des recherches	Obligatoire
Tableaux de bord	KPI de la page d'accueil, graphiques Chart.js, visualisations et dashboard Kibana	Obligatoire
Intermédiaire	Authentification et rôles, cache Redis, API REST documentée avec Swagger	Intermédiaire
Alerting (G)	Règles (temps de réponse > 3 s, pic de 5xx, absence de logs), worker, cooldown, notifications email/webhook, suivi des alertes	Avancé
Temps réel (H)	Ingestion continue, WebSocket, page Live Logs, compteurs en direct	Avancé

1.5 Besoins non fonctionnels

Critère	Exigence
Performance	Logs consultables moins de 10 s après l'import ; recherche en moins de 2 s ; affichage temps réel en moins de 2 s
Disponibilité	Health checks Docker, ordre de démarrage maîtrisé, données persistantes grâce aux volumes
Fiabilité	Une ligne mal formée n'arrête pas l'ingestion ; messages clairs si un service est injoignable
Scalabilité	Index logs-saas-* et architecture permettant d'ajouter des nœuds Elasticsearch

Critère	Exigence
Sécurité	Accès authentifié par rôle, mots de passe hachés, secrets dans .env, rate limiting, contrôle des fichiers importés
Ergonomie	Interface responsive (ordinateur, tablette, mobile), en français, navigation claire
Maintenabilité	Déploiement en une commande (docker compose up), code versionné sur GitHub, README et tests

1.6 Contraintes techniques

Elasticsearch, Logstash et Kibana 8.x · MongoDB 7 · Redis 7 · Flask 3 (Python 3.11) · Bootstrap 5, Chart.js, DataTables · Docker et Docker Compose · GitHub et Jira.

2. Diagramme de cas d'utilisation

Le diagramme présente les fonctionnalités offertes par SaaSWatch et les acteurs qui les utilisent. Tous les acteurs doivent s'authentifier. Le **système de notification** (email, webhook) est un acteur externe qui reçoit les alertes.

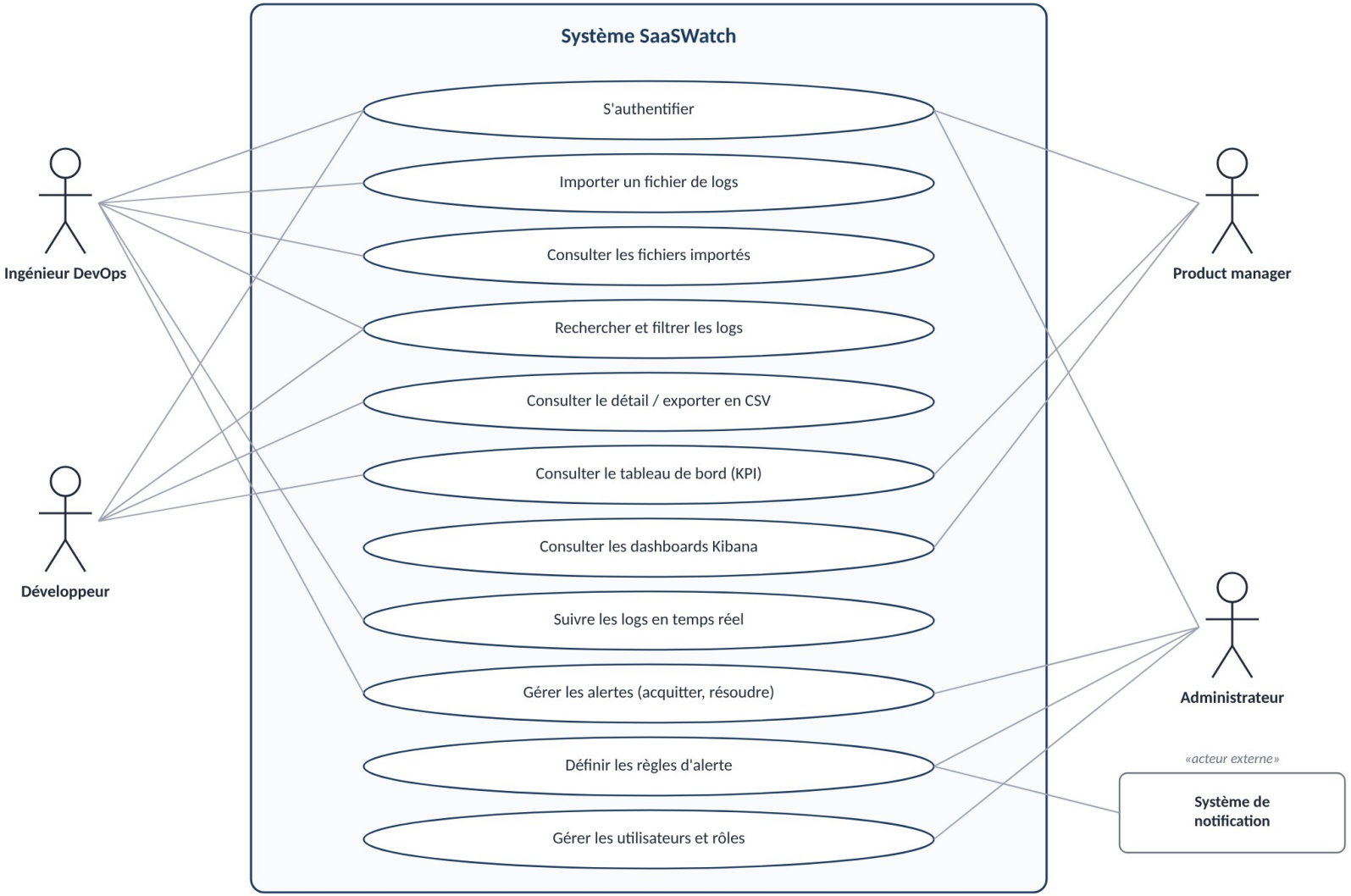


Figure 1 : Diagramme de cas d'utilisation de SaaSWatch

3. Architecture technique

3.1 Architecture globale (C4 – niveau conteneurs)

L'application est composée de services indépendants, tous déployés avec Docker Compose sur un même réseau. Chaque service a un rôle précis :

Service	Rôle
Application Flask	Interface web, API REST /api/v1 et WebSocket pour le temps réel
Logstash	Lit les fichiers importés et le flux continu, transforme les logs et les envoie à Elasticsearch
Elasticsearch	Stocke et indexe les logs (index logs-saas-*), exécute les recherches et agrégations
Kibana	Visualisations et dashboard d'analyse
MongoDB	Métadonnées des fichiers, historique des recherches, utilisateurs, règles et alertes
Redis	Cache des recherches, sessions, rate limiting, cooldown et Pub/Sub des alertes
Worker d'alerting	Évalue périodiquement les règles et déclenche les notifications
Générateur de logs	Script Python Faker qui simule le trafic de l'application SaaS

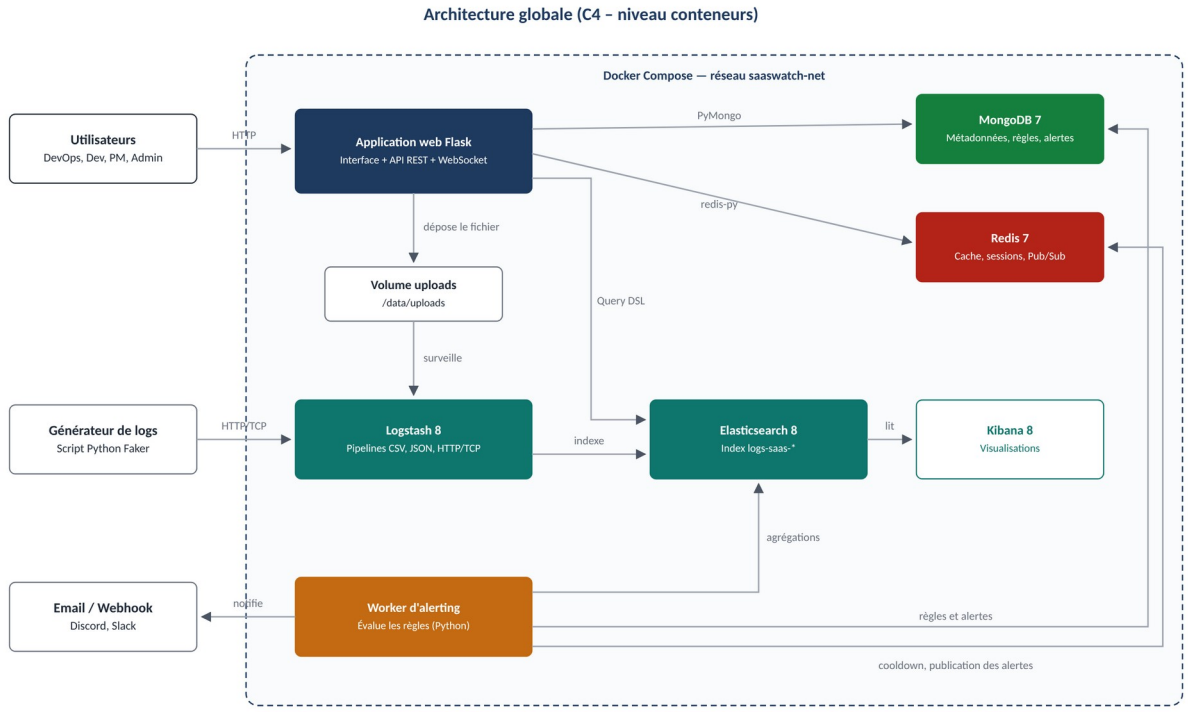


Figure 2 : Architecture globale de SaaSWatch

3.2 Diagramme de déploiement Docker

Chaque service tourne dans son propre conteneur. Les données sont conservées dans des volumes persistants, et la configuration sensible est placée dans un fichier `.env` non versionné.

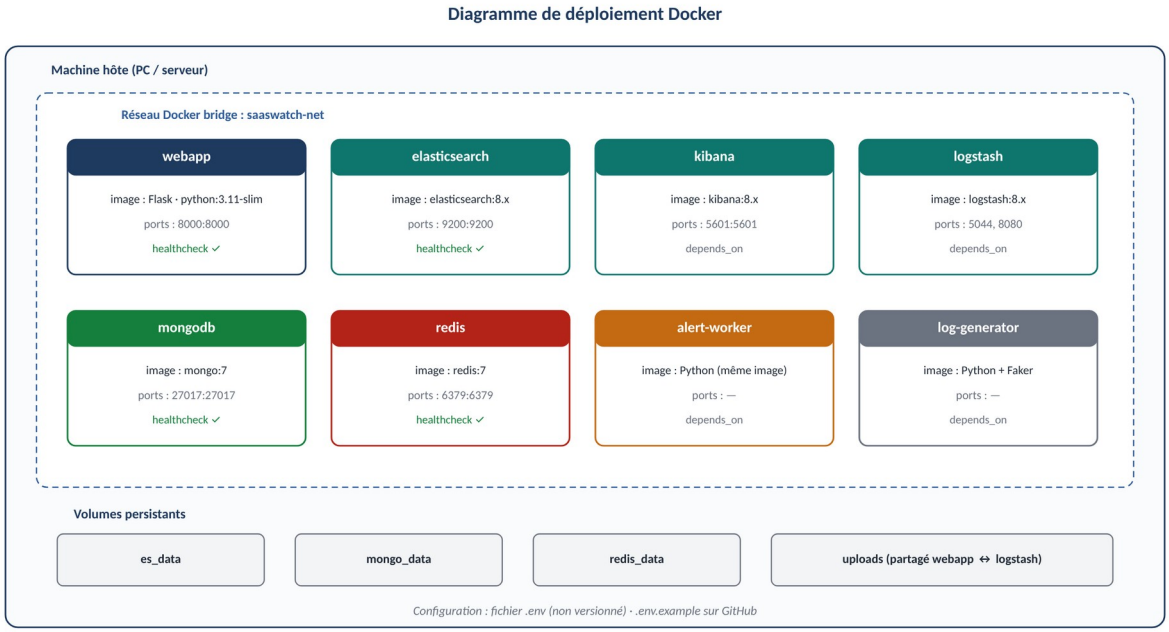


Figure 3 : Diagramme de déploiement Docker

4. Diagrammes de séquence

4.1 Import d'un fichier de logs

L'application valide le fichier, enregistre ses métadonnées dans MongoDB puis le dépose dans le volume surveillé par Logstash. Logstash transforme les lignes et les indexe dans Elasticsearch ; l'application suit l'avancement et met à jour le statut du fichier.

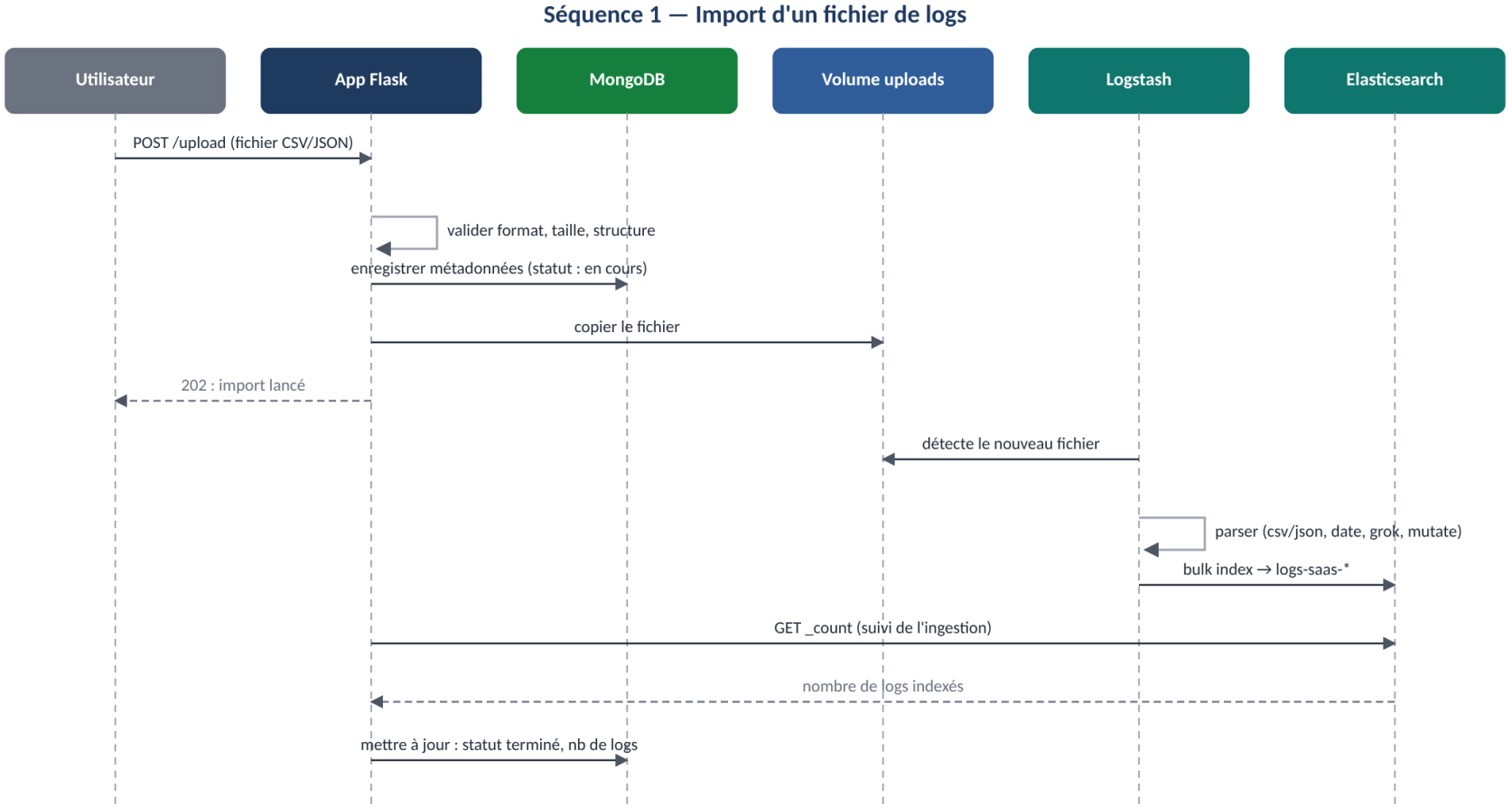


Figure 4 : Séquence d'import d'un fichier

4.2 Recherche de logs

Avant d'interroger Elasticsearch, l'application vérifie si le résultat est déjà dans le cache Redis. La recherche est ensuite enregistrée dans l'historique MongoDB.

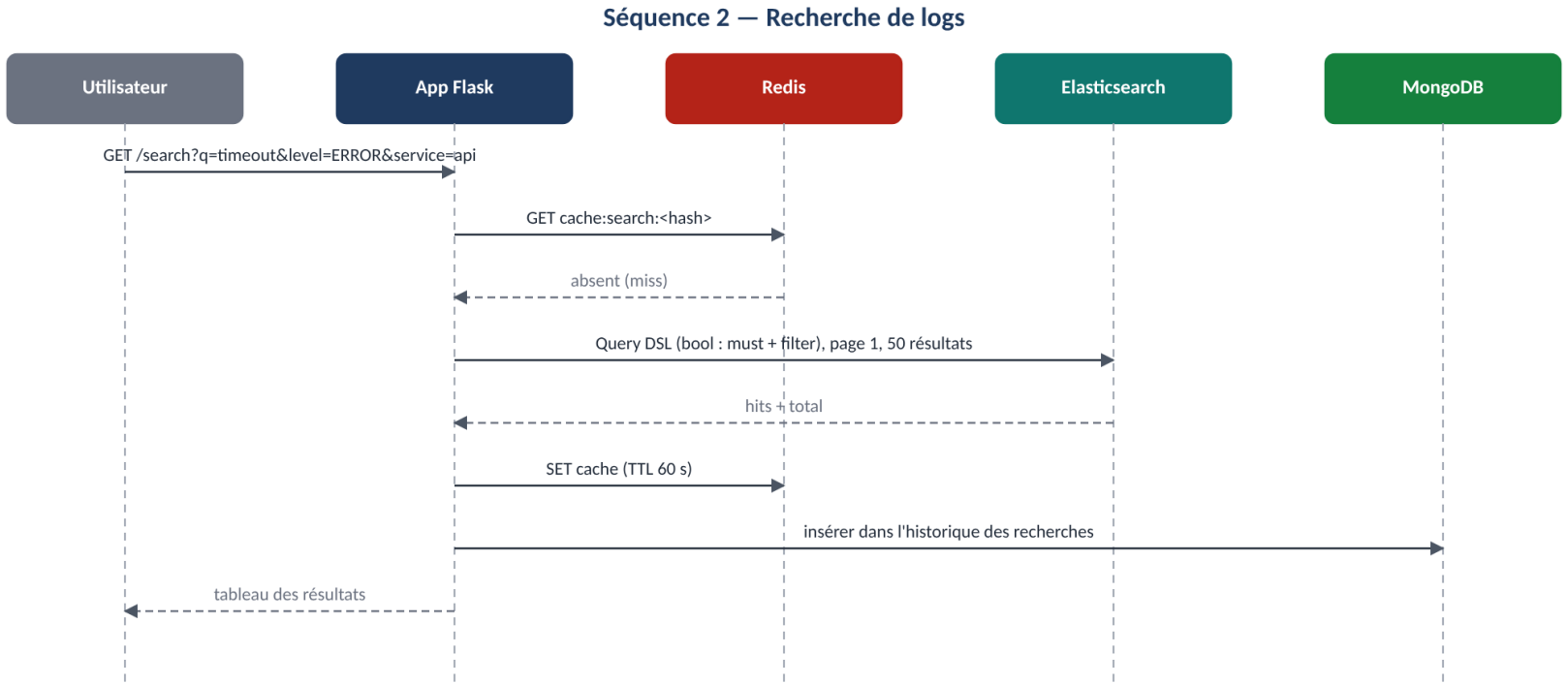


Figure 5 : Séquence de recherche

4.3 Déclenchement d’une alerte

Le worker lit les règles actives, calcule l’indicateur avec une agrégation Elasticsearch et compare la valeur au seuil. Le cooldown stocké dans Redis empêche d’envoyer plusieurs fois la même alerte.

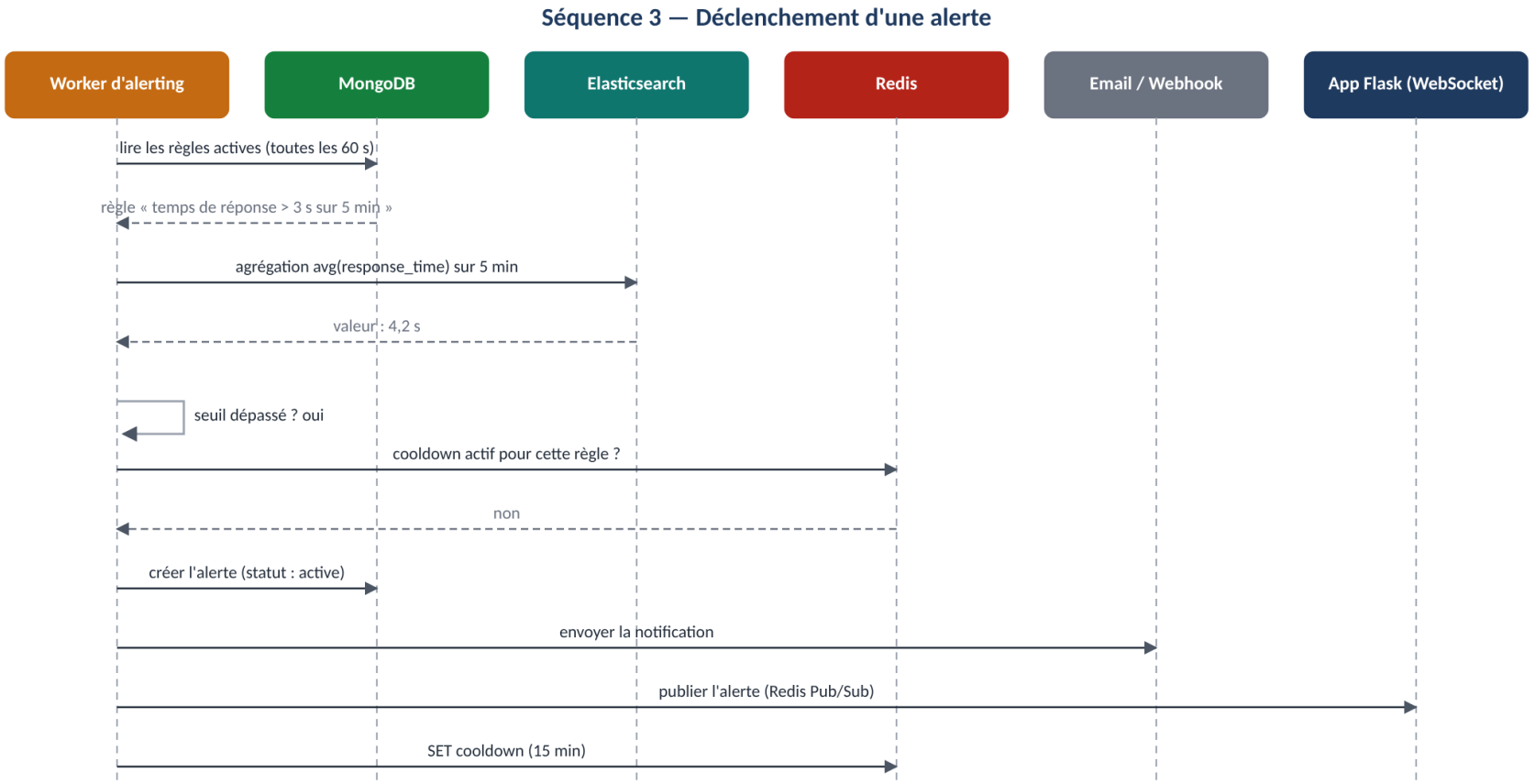


Figure 6 : Séquence de déclenchement d’une alerte

5. Modèle de données

5.1 Elasticsearch : index template logs-saas-*

Les champs servant aux filtres et regroupements sont de type `keyword` ; le message est de type `text` pour la recherche plein texte.

```
PUT _index_template/logs-saas
{
  "index_patterns": ["logs-saas-*"],
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
    "mappings": {
      "properties": {
        "@timestamp":    { "type": "date" },
        "level":         { "type": "keyword" },
        "service":       { "type": "keyword" },
        "source":        { "type": "keyword" },
        "http_method":   { "type": "keyword" },
        "endpoint":      { "type": "keyword" },
        "status_code":   { "type": "integer" },
        "response_time": { "type": "float" },
        "user_id":       { "type": "keyword" },
        "tenant":        { "type": "keyword" },
        "feature":       { "type": "keyword" },
        "client_ip":     { "type": "ip" },
        "sql_query":     { "type": "text" },
        "message":       { "type": "text" },
        "file_id":       { "type": "keyword" }
      }
    }
  }
}
```

Exemple de document indexé :

```
{
  "@timestamp": "2026-11-02T14:05:12Z", "level": "ERROR", "service": "billing",
  "source": "nginx", "http_method": "GET", "endpoint": "/api/invoices",
  "status_code": 500, "response_time": 3.84, "user_id": "u_1042",
  "tenant": "acme", "feature": "invoices", "client_ip": "41.226.12.8",
  "message": "Internal Server Error while generating invoice"
}
```

5.2 MongoDB : collections

Collection	Contenu	Champs principaux
files	Métadonnées des fichiers importés	name, size, type, status, logs_count, uploaded_by, uploaded_at
search_history	Historique des recherches	user_id, query, filters, results_count, created_at
users	Comptes et rôles	username, email, password_hash, role
alert_rules	Règles d'alerte	name, type, condition, window, severity, enabled, cooldown
alerts	Alertes déclenchées	rule_id, value, status, triggered_at, resolved_at, comments

Exemple de document de la collection `alert_rules` :

```
{
  "_id": "6730f1...",
  "name": "Temps de réponse élevé",
  "type": "threshold",
  "condition": { "field": "response_time", "agg": "avg", "operator": ">", "value": 3 },
  "window": "5m",
  "severity": "CRITICAL",
  "cooldown_minutes": 15,
  "channels": ["email", "discord"],
  "enabled": true
}
```

5.3 Redis : clés et usages

Clé	Type	Usage	Durée de vie
cache:search:<hash>	String (JSON)	Résultat d'une recherche	60 s
cache:stats	Hash	KPI de la page d'accueil	30 s
session:<id>	Hash	Session utilisateur	24 h
ratelimit:<user>:<minute>	Compteur	Limitation des appels API	60 s
alert:cooldown:<rule_id>	String	Évite les alertes en double	15 min
channel:alerts / channel:logs	Pub/Sub	Diffusion temps réel vers le WebSocket	—

6. Maquettes de l’interface

6.1 Wireframes

Les wireframes définissent l’organisation de chaque page, sans couleurs. Ils couvrent les 5 pages principales exigées, plus la page Live Logs du module temps réel.

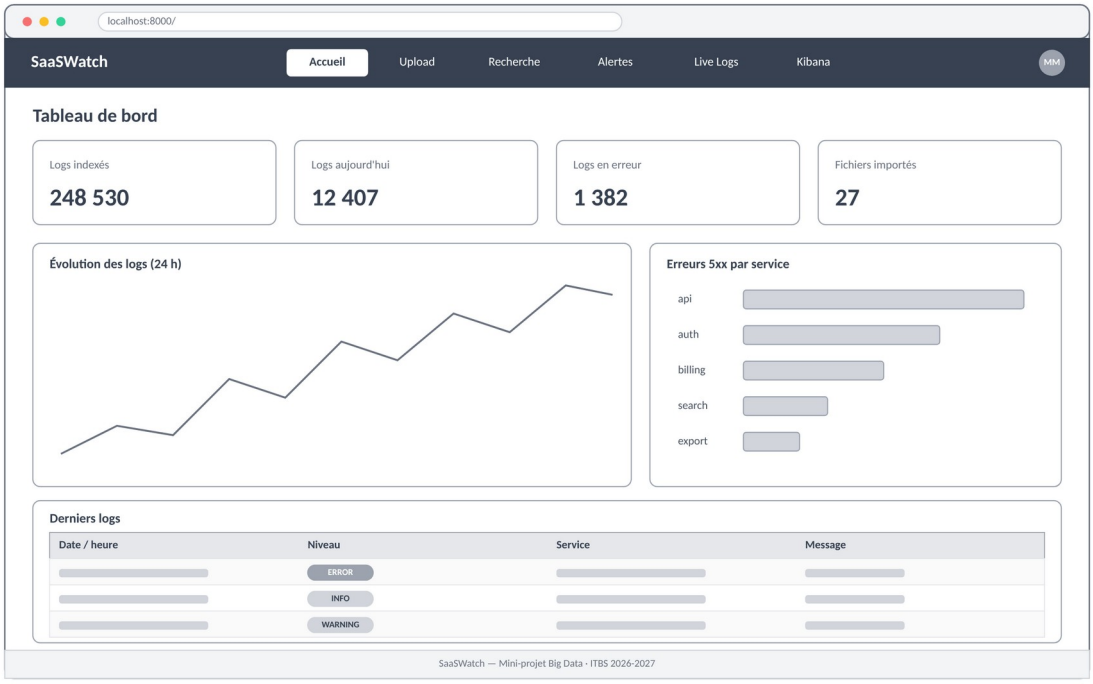


Figure 7 : Wireframe — Tableau de bord

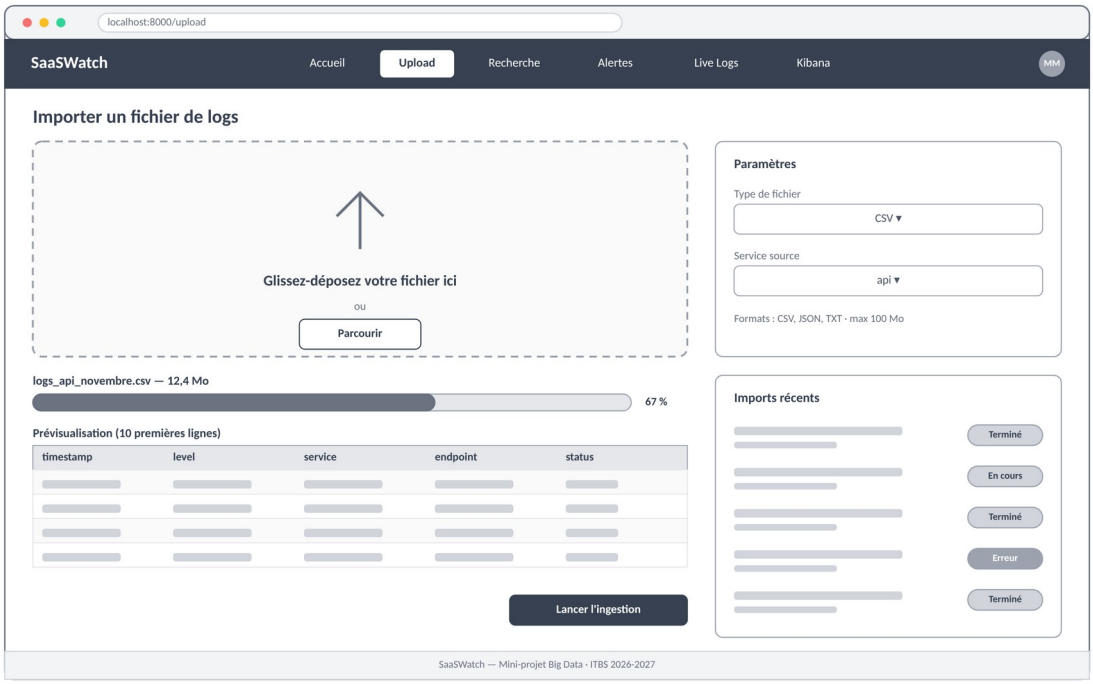


Figure 8 : Wireframe — Import de fichiers

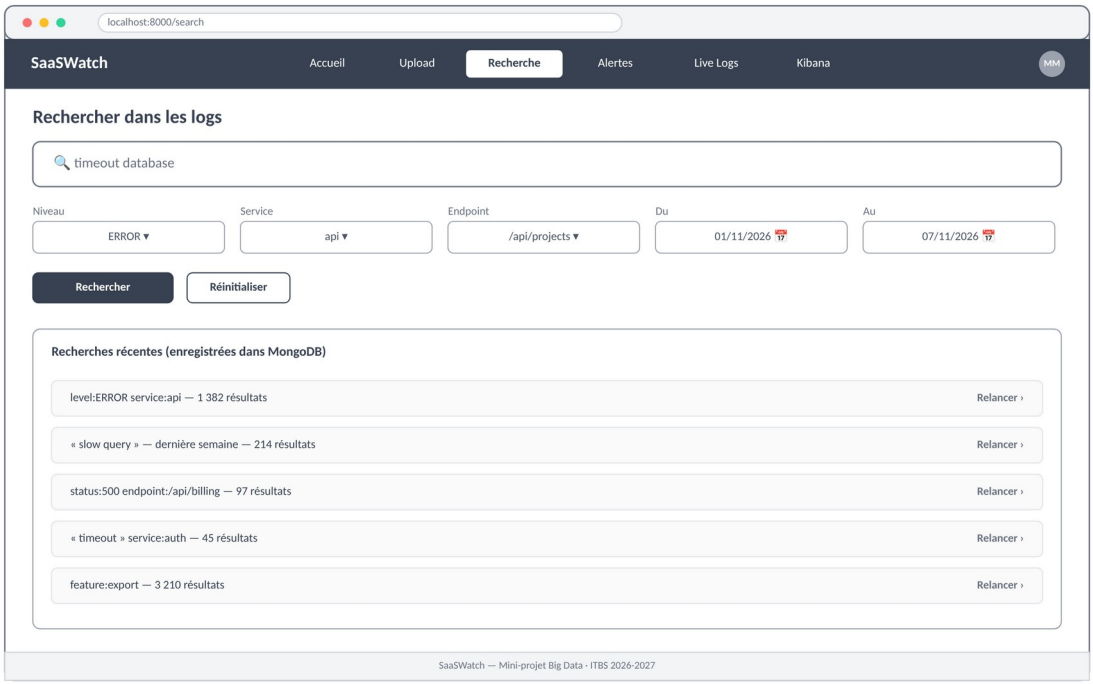


Figure 9 : Wireframe — Recherche

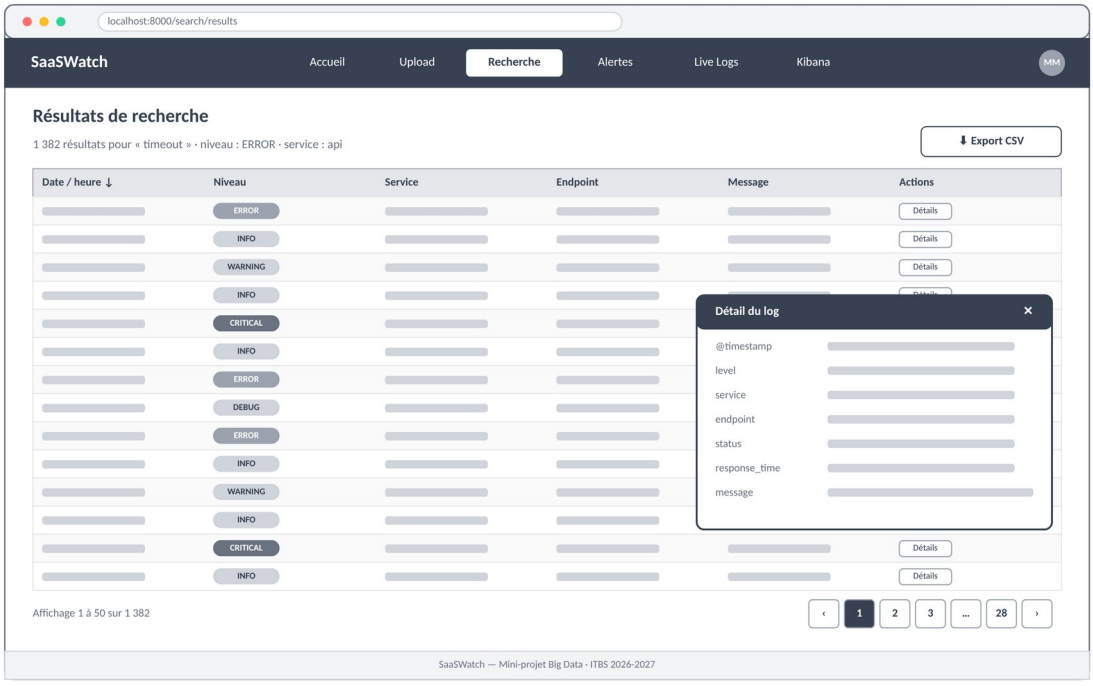


Figure 10 : Wireframe — Résultats et détail d'un log

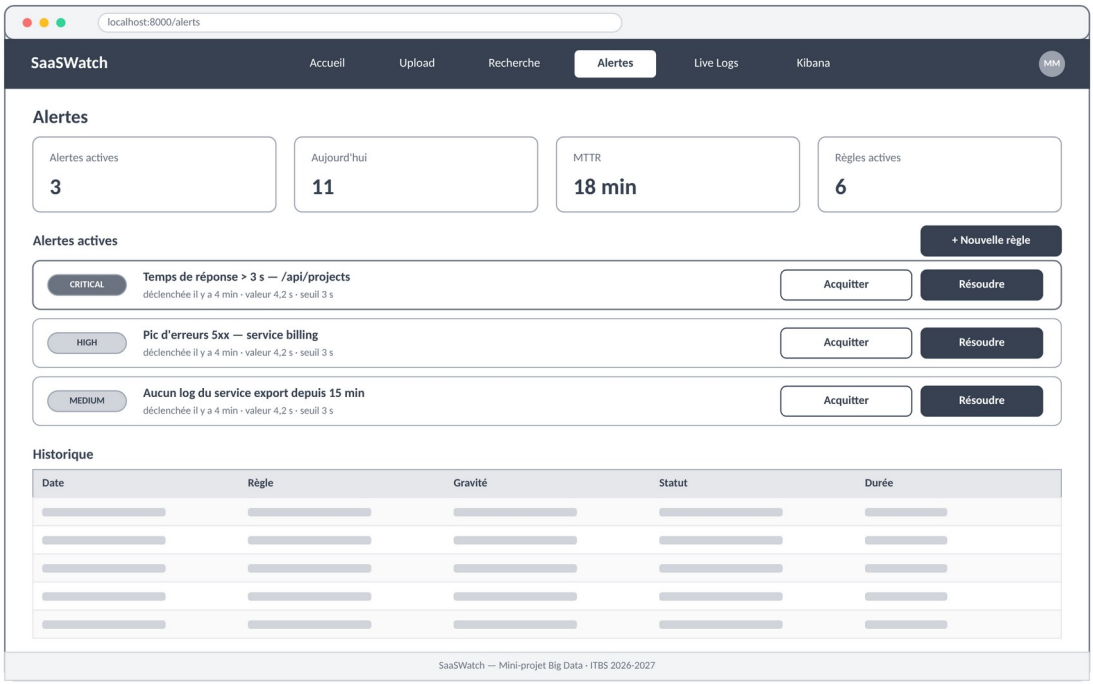


Figure 11 : Wireframe — Alertes

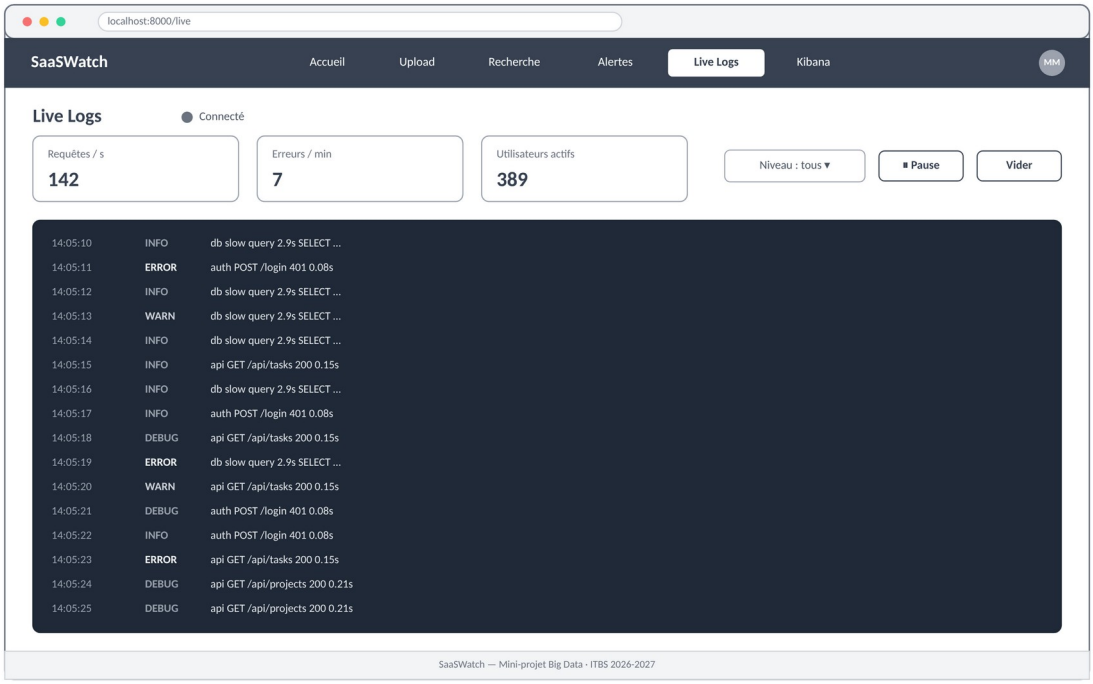


Figure 12 : Wireframe — Live Logs

6.2 Design system

Le design system fixe les couleurs, la typographie et les composants communs à toutes les pages. Chaque niveau de log a une couleur fixe pour être reconnu immédiatement.

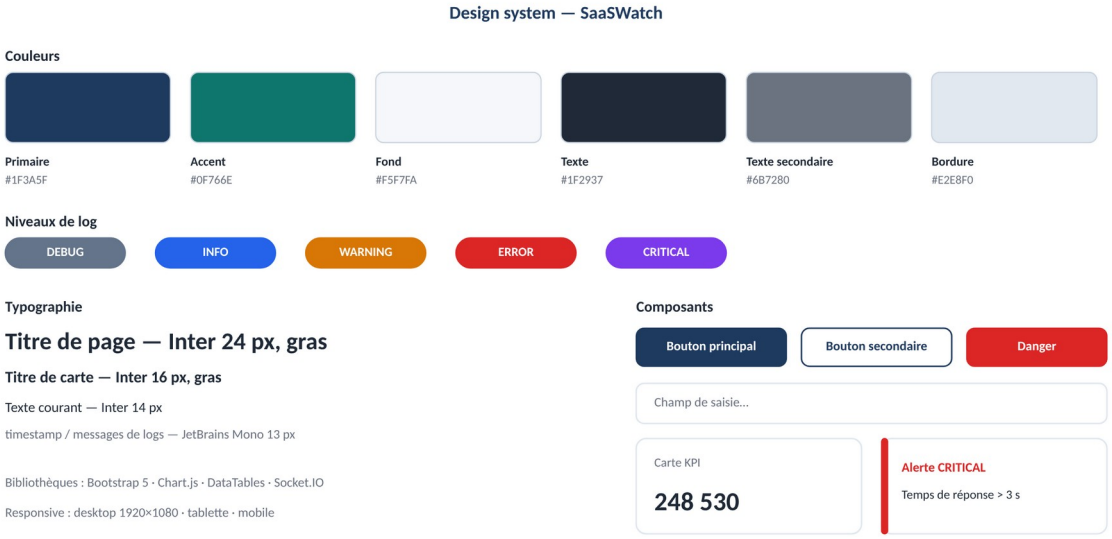


Figure 13 : Design system de SaaSWatch

6.3 Maquette finale : tableau de bord

La maquette applique le design system à la page d'accueil. La version finale place la navigation dans un menu latéral, plus adapté à une application de supervision.

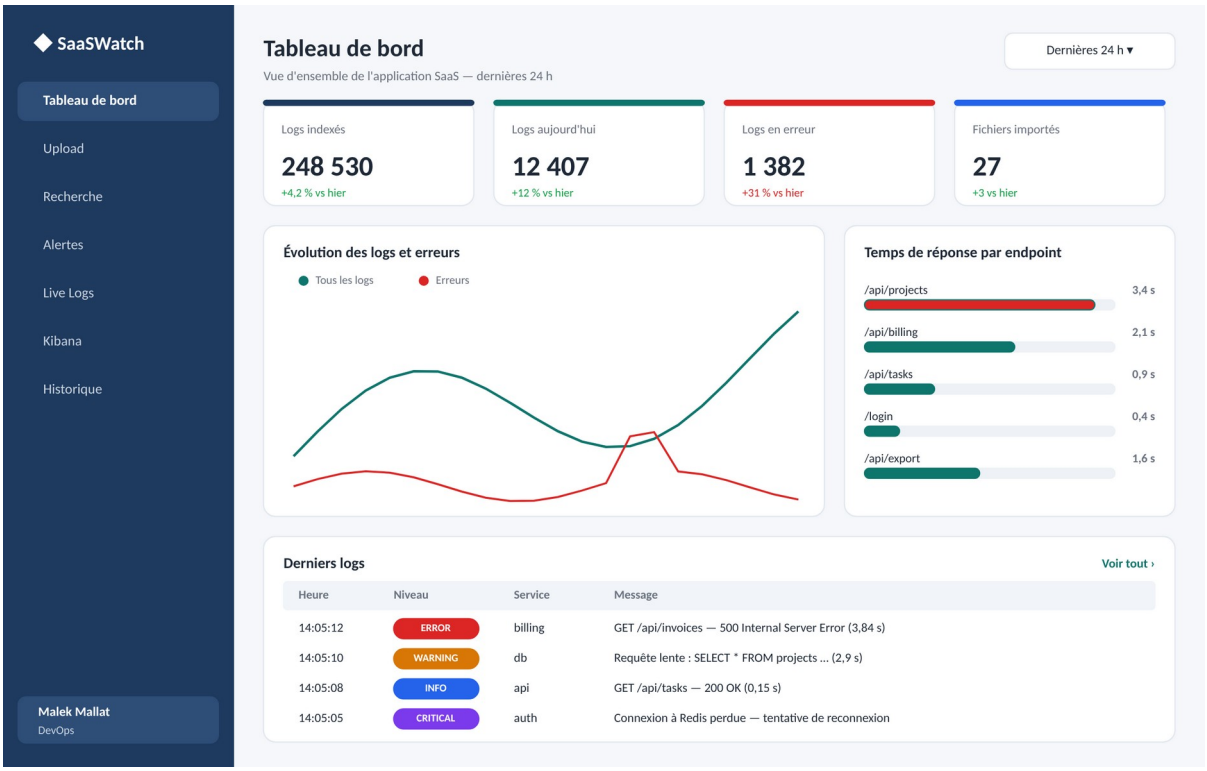


Figure 14 : Maquette du tableau de bord

6.4 Parcours utilisateur

Parcours type d'un ingénieur DevOps lors d'un incident :

- **Notification** : il reçoit une alerte « Temps de réponse > 3 s » sur Discord.
- **Alertes** : il ouvre la page Alertes et acquitte l'incident.
- **Recherche** : il filtre les logs du service concerné sur les 15 dernières minutes.

- **Détail** : il consulte le log en erreur et identifie la requête SQL lente.
- **Live Logs** : il vérifie en direct que les temps de réponse redeviennent normaux, puis marque l'alerte comme résolue.